Weather Station Project Report

By: Alan O'Connell

Lecturer: Donal O'Donovan

Module: Interface Software Development

Date: 17/04/2026

I acknowledge, by submitting this report, that I am aware of the provisions in MTU's Student Regulations which specifically cover cheating where any attempt is made to gain unfair advantage, whether in coursework, assessments or examinations.

I further acknowledge that I understand the Policy and associated Procedures together with the Supplementary Policy and Supplementary Procedure which have been approved for this period

Contents

1. Statement of Complete Specification 1
2. Specification Breakdown with Code Segments 1
2.1 Scan folders and populate ListBoxes 1
2.2 Double-click selects parameter file 2
2.3 Extract time and build key/value pairs 3
2.4 Place parameter name in label 4
2.5 Plot button displays two series on chart 4
2.6 Count label and slider 6
2.7 Range button shows subset on separate chart 7
2.8 Write differences to file 7
2.9 Menu options 8
3. Program Testing 9
3.1 Screenshots 10
4. Code Appendix 11
4.1 MainWindow.xaml 11
4.2 MainWindow.xaml.cs 15
4.3 App.xaml 22
4.4 App.xaml.cs 22

1. Statement of Complete Specification

This application is a WPF C# program that compares weather data from two Met Éireann stations: Sherkin Island and Roches Point. It reads temperature, wind speed, and rainfall data from text files, displays the values side by side using LiveCharts.Wpf (v0.9.7), and writes the differences at each timestamp to a .txt file.

The project follows the specification provided in the brief document, and all required features listed in the specification table below are fully working. Most, if not all, of the code used in this project is based on material covered in the lecture slides and notes. Any external sources used are clearly referenced with a URL in the code comments where applicable.

A GitHub repository for this project is available at: https://github.com/Alan64578/Interface-Dev-Weather-Monitor

Specification Item	Marks	Status
Scan folders, construct file list in ListBoxes (no Time)	14	Complete
Double-click item selects parameter file for extraction	12	Complete
Extract time/date, build key/value pairs for parameter	14	Complete
Place parameter name in label	4	Complete
Plot button displays values on chart, two series	14	Complete
Determine max_element, Count label on screen	2	Complete
Slider adjusts Count between 1 and max_element	16	Complete
Range button shows Count elements on separate chart	10	Complete
Write date-time / parameter differences to file	6	Complete
Menu options: Load Stations, Plot Stations, Exit	8	Complete

2. Specification Breakdown with Code Segments

2.1 Scan folders and populate ListBoxes

Clicking Load Data clears any old data and runs LoadStation for each folder. LoadStation calls Directory.GetFiles to grab every .txt file, loops through them with a for loop, and skips anything ending in "time" since the spec says to leave that out of the ListBox. It strips the station name off the front of each filename, so the list just shows "Temperature", "Speed" and "Rainfall".

There was a bug in the progress report where clicking Load Data a second time left the old file paths in memory. Clearing _sherkinParamFile and _rochesParamFile at the top of the handler fixed that.

XAML (ListBox inside Border with coloured top edge):

```
<Border BorderBrush="#4A9EF5" BorderThickness="0,3,0,0"
      Margin="0,0,0,8" Background="#2A2A2A">
   <StackPanel>
      <TextBlock Text="Sherkin Station"
            FontWeight="Bold" FontSize="13" Foreground="White"
            Margin="10,8,10,4"/>
```

```
      <ListBox x:Name="LstSherkin"
            Margin="8,0,8,6" Height="80"
            Background="#1E1E1E" Foreground="#CCCCCC"
            BorderBrush="#555555" FontSize="12"
            MouseDoubleClick="LstSherkin_MouseDoubleClick"/>

      <StackPanel Orientation="Horizontal" Margin="10,0,10,8">
         <TextBlock Text="Selected: " Foreground="#888888" FontSize="11"/>
         <TextBlock x:Name="TxtSherkinParam" Text=""
               Foreground="#4A9EF5" FontSize="11"/>
      </StackPanel>
   </StackPanel>
</Border>
```

C# LoadStation method:

```
// scans folder and fills the listbox, leaves out Time
private void LoadStation(string folder, ListBox listBox, string stationPrefix)
{
   if (!Directory.Exists(folder))
   {
      MessageBox.Show("Folder not found:\n" + folder,
         "Missing Folder", MessageBoxButton.OK, MessageBoxImage.Warning);
      return;
   }

   listBox.Items.Clear();

   string[] files = Directory.GetFiles(folder, "*.txt");

   for (int i = 0; i < files.Length; i++)
   {
      string fileName = Path.GetFileNameWithoutExtension(files[i]);

      // dont put Time in the listbox
      if (fileName.ToLower().EndsWith("time"))
         continue;

      string paramName = fileName.Replace(stationPrefix, "").Trim();
      listBox.Items.Add(paramName);
   }
}
```

Directory.Exists checks the folder is there before trying to read from it. If it is missing the user gets a MessageBox with the file path. The ToLower().EndsWith("time") call filters out the Time file. Replace strips the prefix so the ListBox shows "Temperature" instead of "Sherkin Temperature".

2.2 Double-click selects parameter file

Both ListBoxes have a MouseDoubleClick handler. Double-clicking an item builds the full path to that parameter file by combining the folder path, station name and parameter name. The path goes into a string field and the "Selected" label updates. After that it calls CheckReady, which turns on the Plot button once both stations have a file path that exists on disk.

C#:

```
private void LstSherkin_MouseDoubleClick(object sender,
   System.Windows.Input.MouseButtonEventArgs e)
{
   if (LstSherkin.SelectedItem == null) return;

   string param = LstSherkin.SelectedItem.ToString();
```

```
    _sherkinParamFile = Path.Combine(_sherkinFolder, "Sherkin " + param + ".txt");
    TxtSherkinParam.Text = param;

    CheckReady();
}

// enables Plot once both files are picked
private void CheckReady()
{
    bool ready = _sherkinParamFile != "" &&
                 _rochesParamFile != "" &&
                 File.Exists(_sherkinParamFile) &&
                 File.Exists(_rochesParamFile);

    BtnPlot.IsEnabled = ready;
    MenuPlotStations.IsEnabled = ready;
}
```

The null check at the top stops it crashing or doing unintentional actions, if the user double-clicks on empty space in the ListBox. CheckReady calls File.Exists on both paths before enabling Plot. The Roches handler is the same code with different folder and field names.

2.3 Extract time and build key/value pairs

ExtractData opens the Time file and the parameter file at the same time using two StreamReaders. It reads them line by line in a while loop. For each pair of lines, it sticks the date onto the front of the time string, parses that into a DateTime, converts the value to a double, and drops both into a SortedDictionary. The dictionary keeps everything in time order which is what the chart needs.

C#:

```
private SortedDictionary<DateTime, double> ExtractData(string timeFile, string paramFile)
{
    SortedDictionary<DateTime, double> result = new SortedDictionary<DateTime, double>();
    StreamReader srTime = null;
    StreamReader srValues = null;

    try
    {
        srTime = new StreamReader(timeFile);
        srValues = new StreamReader(paramFile);

        string timeLine;
        string valueLine;

        while ((timeLine = srTime.ReadLine()) != null &&
               (valueLine = srValues.ReadLine()) != null)
        {
            // stick todays date on the front
            string dateTimeStr = DateTime.Today.ToString("yyyy-MM-dd") + " " + timeLine.Trim();

            try
            {
                DateTime dt = DateTime.Parse(dateTimeStr);
                double val = Convert.ToDouble(valueLine.Trim());
                result[dt] = val;
            }
            catch (FormatException)
            {

            }
        }
    }
```

```
    catch (FileNotFoundException ex)
    {
        MessageBox.Show("File not found:\n" + ex.Message,
            "Missing File", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error reading data:\n" + ex.Message,
            "Read Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
    finally
    {
        if (srTime != null) srTime.Close();
        if (srValues != null) srValues.Close();
    }

    return result;
}
```

A try/catch/finally wraps the whole method so the finally block can close both StreamReaders even if something goes wrong halfway through. Inside the while loop there is a second try/catch around DateTime.Parse and Convert.ToDouble. One bad line in the file won't kill the entire read, it just gets skipped. FileNotFoundException has its own catch block so the error message can name the missing file.

2.4 Place parameter name in label

When you double-click a parameter, the handler puts the name into TxtSherkinParam.Text (or TxtRochesParam for the other station). In the XAML there is a grey "Selected:" label next to a coloured one that shows whatever text gets set.

C# (inside double-click handler):

```
TxtSherkinParam.Text = param;
```

XAML:

```
<StackPanel Orientation="Horizontal" Margin="10,0,10,8">
    <TextBlock Text="Selected: " Foreground="#888888" FontSize="11"/>
    <TextBlock x:Name="TxtSherkinParam" Text=""
            Foreground="#4A9EF5" FontSize="11"/>
</StackPanel>
```

2.5 Plot button displays two series on chart

The Plot handler starts with two checks. First, it makes sure both stations have a file path set. Then it compares the parameter names to stop the user plotting Temperature against Rainfall on the same axis. After that it calls ExtractData for both stations, pulls the values into List<double> collections, and creates two LineSeries with ChartValues<double>. It builds a new SeriesCollection each time and assigns it to ChartPlot.Series.

After plotting, it works out the slider range and calls WriteDifferencesFile so the differences file gets written straight away.

C# (key parts of Plot handler):

```
// cant plot temperature vs rainfall on same axis
if (TxtSherkinParam.Text.ToLower() != TxtRochesParam.Text.ToLower())
{
```

```
    MessageBox.Show("Please select the same parameter in both listboxes.");
    return;
}

_sherkinData = ExtractData(sherkinTimeFile, _sherkinParamFile);
_rochesData = ExtractData(rochesTimeFile, _rochesParamFile);

// x-axis labels from sherkin timestamps
PlotLabels.Clear();
foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
{
    PlotLabels.Add(pair.Key.ToString("HH:mm"));
}

List<double> sherkinValues = new List<double>();
foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
{
    sherkinValues.Add(pair.Value);
}

List<double> rochesValues = new List<double>();
foreach (KeyValuePair<DateTime, double> pair in _rochesData)
{
    rochesValues.Add(pair.Value);
}

LineSeries sherkinSeries = new LineSeries
{
    Title = "Sherkin Island",
    Values = new ChartValues<double>(sherkinValues),
    PointGeometrySize = 6,
    StrokeThickness = 2
};

LineSeries rochesSeries = new LineSeries
{
    Title = "Roches Point",
    Values = new ChartValues<double>(rochesValues),
    PointGeometrySize = 6,
    StrokeThickness = 2
};

ChartPlot.Series = new SeriesCollection { sherkinSeries, rochesSeries };

if (ChartPlot.AxisX.Count > 0)
    ChartPlot.AxisX[0].Labels = PlotLabels;

// write differences straight away
WriteDifferencesFile();
```

XAML CartesianChart:

```
<lvc:CartesianChart x:Name="ChartPlot" Grid.Row="1" Margin="4" Background="#1E1E1E">
    <lvc:CartesianChart.AxisX>
        <lvc:Axis Title="Time" Foreground="#AAAAAA">
            <lvc:Axis.Separator>
                <lvc:Separator StrokeThickness="0.3" Stroke="#444444"/>
            </lvc:Axis.Separator>
        </lvc:Axis>
    </lvc:CartesianChart.AxisX>
    <lvc:CartesianChart.AxisY>
        <lvc:Axis Title="Value" Foreground="#AAAAAA">
            <lvc:Axis.Separator>
                <lvc:Separator StrokeThickness="0.3" Stroke="#444444"/>
            </lvc:Axis.Separator>
```

```
        </lvc:Axis>
    </lvc:CartesianChart.AxisY>
</lvc:CartesianChart>
```

A foreach loop takes the SortedDictionary since that is what the course covered. Each time the user clicks Plot, the code builds a fresh SeriesCollection with two new LineSeries. The x-axis labels come from the Sherkin timestamps formatted as HH:mm. Axis.Separator elements in the XAML give faint grid lines on the chart background.

2.6 Count label and slider

Once the data is plotted, the code compares the two dictionary counts and takes whichever is smaller. That becomes the slider maximum and the minimum is 1. Dragging the slider fires SldCount_ValueChanged which writes the current position into the Count label. A small text underneath shows the full range.

C# (inside BtnPlot_Click):

```
// max_element is the smaller count
int maxElement = _sherkinData.Count;
if (_rochesData.Count < maxElement)
    maxElement = _rochesData.Count;

SldCount.Minimum = 1;
SldCount.Maximum = maxElement;
SldCount.Value = 1;
SldCount.IsEnabled = true;
TxtCount.Text = "1";
TxtCountRange.Text = "1 to " + maxElement;
```

C# ValueChanged:

```
private void SldCount_ValueChanged(object sender,
    RoutedPropertyChangedEventArgs<double> e)
{
    if (TxtCount == null) return;

    int count = (int)SldCount.Value;
    TxtCount.Text = count.ToString();
}
```

XAML:

```
<Slider x:Name="SldCount" Minimum="1" Maximum="1"
        IsEnabled="False" Margin="4,0,4,2"
        ValueChanged="SldCount_ValueChanged"/>
```

TxtCount can be null when ValueChanged activates during InitializeComponent, before the controls are created. Without the null check the app crashes on startup. The slider stays greyed out with IsEnabled=false until data is plotted. The brief asks for a "potentiometer control (or equivalent)" and a WPF Slider does this job and looks better than other options available.

2.7 Range button shows subset on separate chart

The Range handler reads the slider value and grabs the first entries from each SortedDictionary. It builds new List<double> collections for each station, creates two fresh LineSeries, and assigns a new SeriesCollection to ChartRange.Series

C# (key part of Range handler):

```
int count = (int)SldCount.Value;

List<DateTime> sherkinKeys = new List<DateTime>();
List<double> sherkinValues = new List<double>();
int n = 0;
foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
{
    if (n >= count) break;
    sherkinKeys.Add(pair.Key);
    sherkinValues.Add(pair.Value);
    n++;
}

List<double> rochesValues = new List<double>();
n = 0;
foreach (KeyValuePair<DateTime, double> pair in _rochesData)
{
    if (n >= count) break;
    rochesValues.Add(pair.Value);
    n++;
}

RangeLabels.Clear();
for (int i = 0; i < sherkinKeys.Count; i++)
{
    RangeLabels.Add(sherkinKeys[i].ToString("HH:mm"));
}

ChartRange.Series = new SeriesCollection { sherkinRange, rochesRange };
```

The Range handler follows the same approach as Plot. The SortedDictionary is already in time order, so the first n entries are the earliest readings.

2.8 Write differences to file

WriteDifferencesFile goes through the Sherkin dictionary and uses ContainsKey to find timestamps that also exist in Roches. For each match it subtracts Roches from Sherkin and writes a tab-separated line with the time, both values and the difference. Same try/catch/finally pattern as the StreamReader code.

C# (key part):

```
List<DateTime> commonKeys = new List<DateTime>();
foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
{
    if (_rochesData.ContainsKey(pair.Key))
    {
        commonKeys.Add(pair.Key);
    }
}

string paramName = TxtSherkinParam.Text;
string diffFile = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
    paramName + "_Differences.txt");
```

```
sw = new StreamWriter(diffFile);
sw.WriteLine("DateTime\tSherkin\tRoches\tDifference");

for (int i = 0; i < commonKeys.Count; i++)
{
    DateTime key = commonKeys[i];
    double sVal = _sherkinData[key];
    double rVal = _rochesData[key];
    double diff = sVal - rVal;

    sw.WriteLine(key.ToString("yyyy-MM-dd HH:mm") + "\t" +
        sVal + "\t" + rVal + "\t" + diff.ToString("F2"));
}
```

The file gets named after the parameter, so plotting Temperature gives you Temperature_Differences.txt. It is placed in the same folder as the .exe file. The data folders sit there as well, so keeping the output in the same place made sense. The final block closes the StreamWriter. BtnPlot_Click then calls WriteDifferencesFile at the end, so the file gets written straight after plotting. The user can also trigger it again from the Write Differences menu item.

2.9 Menu options

Four menu items across the top: Load Stations, Plot Stations, Write Differences and Exit Application. Load and Plot point at the same click handlers as the sidebar buttons. Write Differences calls WriteDifferencesFile. Exiting calls Application.Current.Shutdown(). Plot Stations and Write Differences start disabled and get switched on after data is plotted.

XAML:

```
<Menu DockPanel.Dock="Top"  Height="36" Background="#2D2D2D" Foreground="White" FontSize="13">
    <MenuItem Header="Load Stations" Click="BtnLoadData_Click"/>
    <MenuItem Header="Plot Stations" x:Name="MenuPlotStations"
            IsEnabled="False" Click="BtnPlot_Click"/>
    <MenuItem Header="Write Differences" x:Name="MenuWriteDiff"
            IsEnabled="False" Click="MenuWriteDiff_Click"/>
    <MenuItem Header="Exit Application" Click="MenuExit_Click"/>
</Menu>
```

C# Write Differences and Exit handlers:

```
private void MenuWriteDiff_Click(object sender, RoutedEventArgs e)
{
    WriteDifferencesFile();
}

private void MenuExit_Click(object sender, RoutedEventArgs e)
{
    Application.Current.Shutdown();
}
```

Load and Plot reuse the button handlers so there is no duplicated code. MenuWriteDiff_Click calls WriteDifferencesFile, which handles the case where no matching timestamps exist by updating the status bar text. IsEnabled keeps the menu items greyed out until BtnPlot_Click enables them.

3. Program Testing

Twelve tests were run against the finished app. Everything passes. The three failures from the progress report are now fixed.

Test	Expected Result	Result
Click Load Data with both folders present.	Both ListBoxes show Temperature, Speed, Rainfall. No Time.	Pass
Click Load Data when SherkinWeatherData folder is missing.	MessageBox warning with missing folder path.	Pass
Double-click Temperature in Sherkin ListBox.	Selected label updates to Temperature.	Pass
Click Plot with no parameter selected.	MessageBox asks user to select parameters.	Pass
Select Temperature in both, click Plot.	Both series on Parameter Chart, time on X-axis.	Pass
Check point count on chart matches file lines.	Chart data points match line count in data file.	Pass
Select Temperature in Sherkin only, click Plot.	Plot stays disabled. No crash.	Pass
Plot data, drag slider to 8, click Range.	Range Chart shows 8 data points.	Pass
Plot data, drag slider to max, click Range.	Range Chart shows full dataset.	Pass
Select Rainfall (Sherkin) and Temperature (Roches), click Plot.	MessageBox says parameters must match.	Pass
After plotting Temperature, check differences file.	Temperature_Differences.txt has correct values.	Pass
Delete a data file from disk, click Plot.	MessageBox shows file error. App does not crash.	Pass

Clearing _sherkinParamFile and _rochesParamFile at the top of BtnLoadData_Click fixed the stale path bug. The slider works now because SldCount_ValueChanged writes the current value into TxtCount.Text every time it fires. For the mismatch bug, both parameter name strings get compared with ToLower() before the plot goes ahead.

3.1 Screenshots

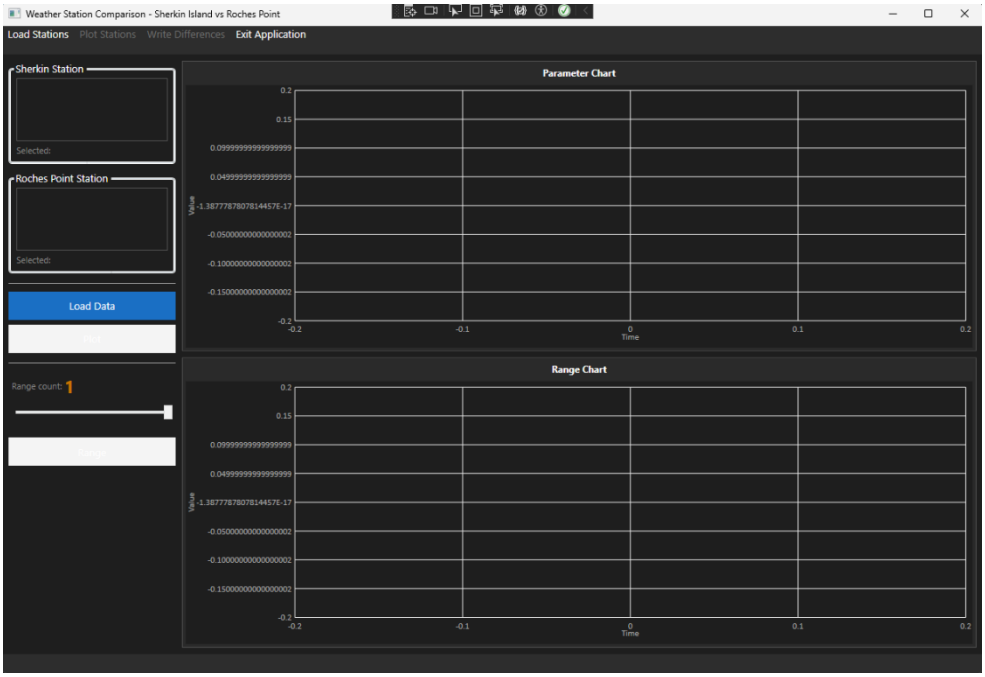

Figure 1: App on startup before loading data. Both ListBoxes are empty. Plot and Range buttons are greyed out. Both charts show empty axes.

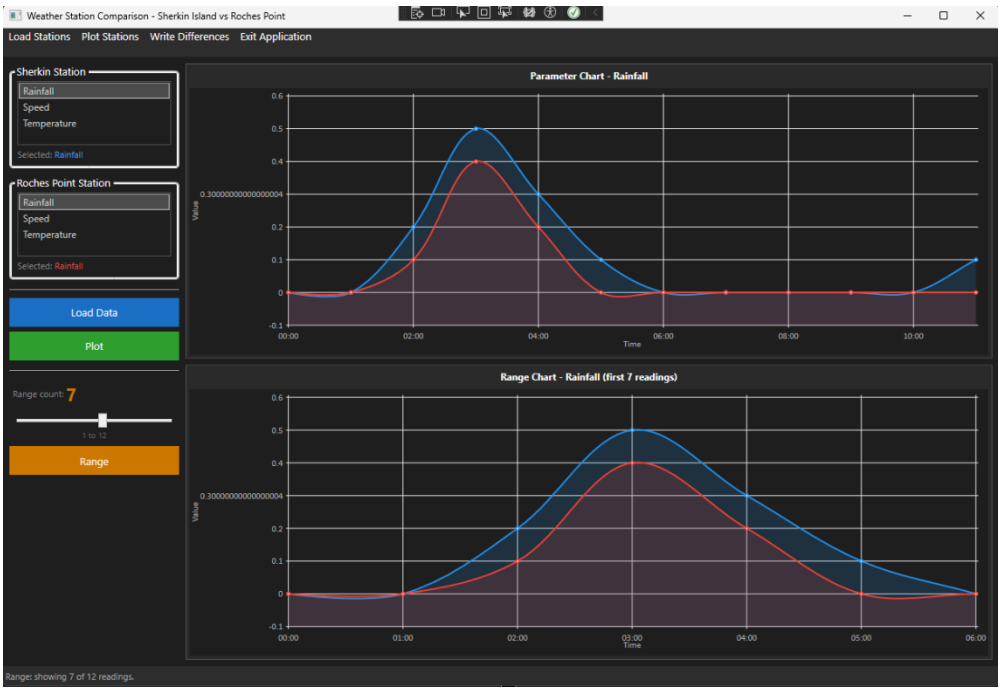

Figure 2: Rainfall plotted for both stations. Sherkin (blue) and Roches Point (red) visible on the Parameter Chart. Slider is at 7 with range 1 to 12. Range Chart below shows the first 7 readings.

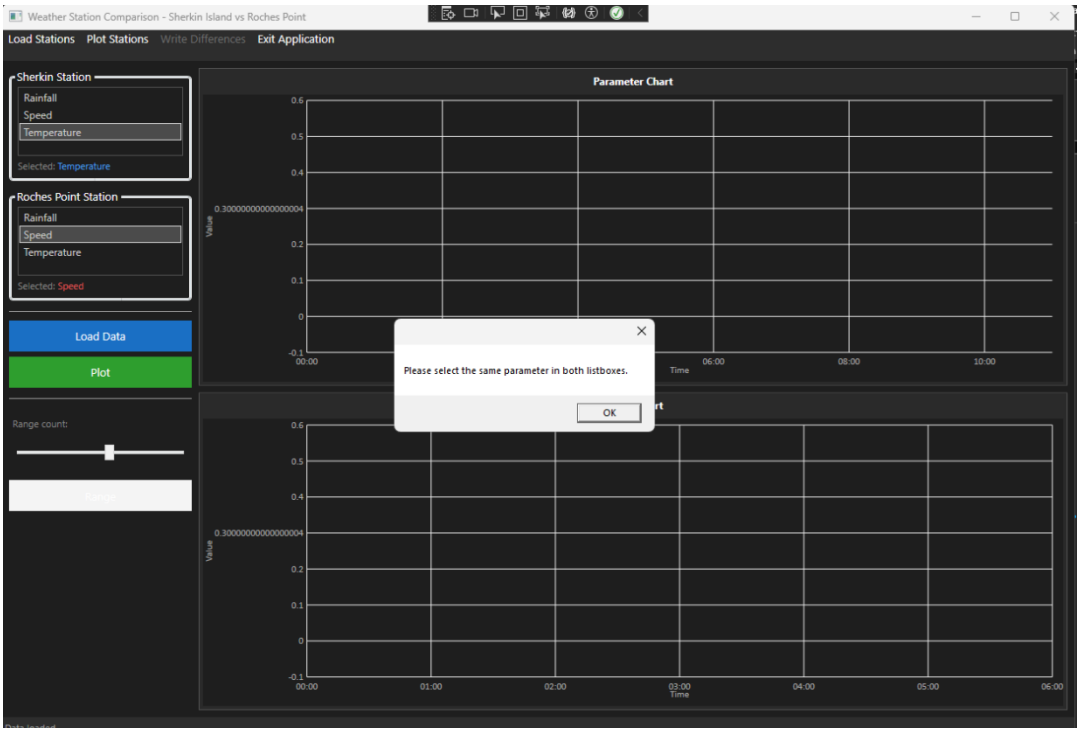

Figure 3: Parameter mismatch. Temperature selected for Sherkin, Speed selected for Roches Point. MessageBox says "Please select the same parameter in both listboxes."

4. Code Appendix

4.1 MainWindow.xaml

```
<Window x:Class="WeatherStation.MainWindow"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      xmlns:lvc="clr-namespace:LiveCharts.Wpf;assembly=LiveCharts.Wpf"
      Title="Weather Station Comparison - Sherkin Island vs Roches Point"
      Height="860" Width="1260"
      Background="#1E1E1E"
      ResizeMode="CanResize">

   <DockPanel>

      <!-- menu bar -->
      <Menu DockPanel.Dock="Top" Height="36" Background="#2D2D2D" Foreground="White" FontSize="13">
         <MenuItem Header="Load Stations" Foreground="White" Background="#2D2D2D"
               Click="BtnLoadData_Click"/>
         <MenuItem Header="Plot Stations" Foreground="White" Background="#2D2D2D"
               x:Name="MenuPlotStations" IsEnabled="False"
               Click="BtnPlot_Click"/>
         <MenuItem Header="Write Differences" Foreground="White" Background="#2D2D2D"
               x:Name="MenuWriteDiff" IsEnabled="False"
               Click="MenuWriteDiff_Click"/>
         <MenuItem Header="Exit Application" Foreground="White" Background="#2D2D2D"
               Click="MenuExit_Click"/>
      </Menu>

      <!-- status bar at the bottom -->
      <StatusBar DockPanel.Dock="Bottom" Height="24" Background="#2D2D2D">
         <StatusBarItem>
```

```
        <TextBlock x:Name="TxtStatus" Text="" Foreground="#AAAAAA" FontSize="11"/>
    </StatusBarItem>
  </StatusBar>

  <!-- 2 columns, sidebar on left and charts on right -->
  <Grid Background="#1E1E1E" Margin="8">
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="220"/>
      <ColumnDefinition Width="*"/>
    </Grid.ColumnDefinitions>

    <!-- sidebar with the two stations and buttons -->
    <StackPanel Grid.Column="0" Margin="0,0,8,0">

      <!-- sherkin station -->
      <GroupBox Header="Sherkin Station" Foreground="White"
            FontSize="13" Margin="0,0,0,8">
        <StackPanel>
          <ListBox x:Name="LstSherkin"
                Margin="4,4,4,4" Height="80"
                Background="#1E1E1E" Foreground="#CCCCCC"
                BorderBrush="#555555" FontSize="12"
                MouseDoubleClick="LstSherkin_MouseDoubleClick"/>

          <StackPanel Orientation="Horizontal" Margin="4,0,4,4">
            <TextBlock Text="Selected: " Foreground="#888888" FontSize="11"/>
            <TextBlock x:Name="TxtSherkinParam" Text=""
                  Foreground="#4A9EF5" FontSize="11"/>
          </StackPanel>
        </StackPanel>
      </GroupBox>

      <!-- roches point station -->
      <GroupBox Header="Roches Point Station" Foreground="White"
            FontSize="13" Margin="0,0,0,8">
        <StackPanel>
          <ListBox x:Name="LstRoches"
                Margin="4,4,4,4" Height="80"
                Background="#1E1E1E" Foreground="#CCCCCC"
                BorderBrush="#555555" FontSize="12"
                MouseDoubleClick="LstRoches_MouseDoubleClick"/>

          <StackPanel Orientation="Horizontal" Margin="4,0,4,4">
            <TextBlock Text="Selected: " Foreground="#888888" FontSize="11"/>
            <TextBlock x:Name="TxtRochesParam" Text=""
                  Foreground="#E05555" FontSize="11"/>
          </StackPanel>
        </StackPanel>
      </GroupBox>

      <Separator Margin="0,4,0,10"/>

      <Button x:Name="BtnLoadData" Content="Load Data"
            Height="36" Margin="0,0,0,6" FontSize="13"
            Background="#1A6FC4" Foreground="White" BorderThickness="0"
            Click="BtnLoadData_Click"/>

      <Button x:Name="BtnPlot" Content="Plot"
            Height="36" Margin="0,0,0,6" FontSize="13"
            Background="#2E9E2E" Foreground="White" BorderThickness="0"
            IsEnabled="False"
            Click="BtnPlot_Click"/>
```

```
    <Separator Margin="0,6,0,10"/>

    <!-- count and slider for range -->
    <StackPanel Margin="0,0,0,6">
      <StackPanel Orientation="Horizontal" Margin="4,4,4,4">
        <TextBlock Text="Range count: " Foreground="#888888" FontSize="11"
                   VerticalAlignment="Center"/>
        <TextBlock x:Name="TxtCount" Text=""
                   Foreground="#CC7700" FontSize="22" FontWeight="Bold"
                   VerticalAlignment="Center"/>
      </StackPanel>
    </StackPanel>

    <Slider x:Name="SldCount" Minimum="1" Maximum="1"
            IsEnabled="False" Margin="4,0,4,2"
            ValueChanged="SldCount_ValueChanged"/>
    <TextBlock x:Name="TxtCountRange" Text=""
               Foreground="#666666" FontSize="10"
               HorizontalAlignment="Center" Margin="0,0,0,8"/>

    <Button x:Name="BtnRange" Content="Range"
            Height="36" Margin="0,0,0,6" FontSize="13"
            Background="#CC7700" Foreground="White" BorderThickness="0"
            IsEnabled="False"
            Click="BtnRange_Click"/>

  </StackPanel>

  <!-- charts go here -->
  <Grid Grid.Column="1">
    <Grid.RowDefinitions>
      <RowDefinition Height="*"/>
      <RowDefinition Height="*"/>
    </Grid.RowDefinitions>

    <!-- main plot chart -->
    <Border Grid.Row="0"
            BorderBrush="#555555" BorderThickness="1"
            Margin="0,0,0,4" Background="#2A2A2A">
      <Grid>
        <Grid.RowDefinitions>
          <RowDefinition Height="Auto"/>
          <RowDefinition Height="*"/>
        </Grid.RowDefinitions>
        <TextBlock x:Name="TxtPlotTitle" Grid.Row="0"
                   Text="Parameter Chart" Foreground="White"
                   FontSize="12" FontWeight="Bold"
                   HorizontalAlignment="Center" Margin="0,6,0,4"/>
        <lvc:CartesianChart x:Name="ChartPlot" Grid.Row="1"
                            Margin="4" Background="#1E1E1E">
          <lvc:CartesianChart.AxisX>
            <lvc:Axis Title="Time" Foreground="#AAAAAA"/>
          </lvc:CartesianChart.AxisX>
          <lvc:CartesianChart.AxisY>
            <lvc:Axis Title="Value" Foreground="#AAAAAA"/>
          </lvc:CartesianChart.AxisY>
        </lvc:CartesianChart>
      </Grid>
    </Border>

    <!-- range chart, shows first N readings -->
```

```
            <Border Grid.Row="1"
                    BorderBrush="#555555" BorderThickness="1"
                    Margin="0,4,0,0" Background="#2A2A2A">
              <Grid>
                <Grid.RowDefinitions>
                  <RowDefinition Height="Auto"/>
                  <RowDefinition Height="*"/>
                </Grid.RowDefinitions>
                <TextBlock x:Name="TxtRangeTitle" Grid.Row="0"
                           Text="Range Chart" Foreground="White"
                           FontSize="12" FontWeight="Bold"
                           HorizontalAlignment="Center" Margin="0,6,0,4"/>
                <lvc:CartesianChart x:Name="ChartRange" Grid.Row="1"
                                    Margin="4" Background="#1E1E1E">
                  <lvc:CartesianChart.AxisX>
                    <lvc:Axis Title="Time" Foreground="#AAAAAA"/>
                  </lvc:CartesianChart.AxisX>
                  <lvc:CartesianChart.AxisY>
                    <lvc:Axis Title="Value" Foreground="#AAAAAA"/>
                  </lvc:CartesianChart.AxisY>
                </lvc:CartesianChart>
              </Grid>
            </Border>

          </Grid>

        </Grid>
      </DockPanel>
    </Window>
```

4.2 MainWindow.xaml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using LiveCharts;
using LiveCharts.Wpf;

namespace WeatherStation
{

    public partial class MainWindow : Window
    {
        // paths to data folders
        private string _sherkinFolder = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory, "SherkinWeatherData");

        private string _rochesFolder = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory, "RochesWeatherData");

        private string _sherkinParamFile = "";
        private string _rochesParamFile = "";

        private SortedDictionary<DateTime, double> _sherkinData = new SortedDictionary<DateTime, double>();
        private SortedDictionary<DateTime, double> _rochesData = new SortedDictionary<DateTime, double>();

        // labels for x-axis on both charts
        public List<string> PlotLabels { get; set; } = new List<string>();
        public List<string> RangeLabels { get; set; } = new List<string>();

        public MainWindow()
        {
            InitializeComponent();
        }

        // resets everything and loads both station folders into the listboxes
        private void BtnLoadData_Click(object sender, RoutedEventArgs e)
        {
            try
            {
                _sherkinParamFile = "";
                _rochesParamFile = "";
                TxtSherkinParam.Text = "";
                TxtRochesParam.Text = "";
                _sherkinData.Clear();
                _rochesData.Clear();

                // https://learn.microsoft.com/en-us/dotnet/api/system.windows.uielement.isenabled
                BtnPlot.IsEnabled = false;
                MenuPlotStations.IsEnabled = false;
                BtnRange.IsEnabled = false;
                SldCount.IsEnabled = false;
                MenuWriteDiff.IsEnabled = false;
                TxtCount.Text = "";
                TxtCountRange.Text = "";

                ChartPlot.Series = null;
                ChartRange.Series = null;
                TxtPlotTitle.Text = "Parameter Chart";
                TxtRangeTitle.Text = "Range Chart";

                LoadStation(_sherkinFolder, LstSherkin, "Sherkin");
                LoadStation(_rochesFolder, LstRoches, "Roches");
```

```
            TxtStatus.Text = "Data loaded.";
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error loading data folders:\n" + ex.Message,
                "Load Error", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    // scans folder and fills the listbox, leaves out Time
    private void LoadStation(string folder, ListBox listBox, string stationPrefix)
    {
        if (!Directory.Exists(folder))
        {
            MessageBox.Show("Folder not found:\n" + folder,
                "Missing Folder", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        listBox.Items.Clear();

        string[] files = Directory.GetFiles(folder, "*.txt");

        for (int i = 0; i < files.Length; i++)
        {
            string fileName = Path.GetFileNameWithoutExtension(files[i]);

            // dont put Time in the listbox
            // https://learn.microsoft.com/en-us/dotnet/api/system.string.tolower
            // https://learn.microsoft.com/en-us/dotnet/api/system.string.endswith
            if (fileName.ToLower().EndsWith("time"))
                continue;

            // https://learn.microsoft.com/en-us/dotnet/api/system.string.replace
            // https://learn.microsoft.com/en-us/dotnet/api/system.string.trim
            string paramName = fileName.Replace(stationPrefix, "").Trim();
            listBox.Items.Add(paramName);
        }
    }

    // when user double clicks a param in the sherkin listbox, store the file path and check if both are ready
    // https://learn.microsoft.com/en-us/dotnet/api/system.windows.controls.control.mousedoubleclick
    private void LstSherkin_MouseDoubleClick(object sender, System.Windows.Input.MouseButtonEventArgs e)
    {
        if (LstSherkin.SelectedItem == null) return;

        string param = LstSherkin.SelectedItem.ToString();
        _sherkinParamFile = Path.Combine(_sherkinFolder, "Sherkin " + param + ".txt");
        TxtSherkinParam.Text = param;

        CheckReady();
    }

    // same as sherkin but for roches point
    private void LstRoches_MouseDoubleClick(object sender, System.Windows.Input.MouseButtonEventArgs e)
    {
        if (LstRoches.SelectedItem == null) return;

        string param = LstRoches.SelectedItem.ToString();
        _rochesParamFile = Path.Combine(_rochesFolder, "Roches " + param + ".txt");
        TxtRochesParam.Text = param;

        CheckReady();
    }

    // enables Plot once both files are picked
```

```
private void CheckReady()
{
    // https://learn.microsoft.com/en-us/dotnet/api/system.io.file.exists
    bool ready = _sherkinParamFile != "" &&
                 _rochesParamFile != "" &&
                 File.Exists(_sherkinParamFile) &&
                 File.Exists(_rochesParamFile);

    BtnPlot.IsEnabled = ready;
    MenuPlotStations.IsEnabled = ready;
}

// reads the time file and param file line by line, pairs them up into a SortedDictionary
private SortedDictionary<DateTime, double> ExtractData(string timeFile, string paramFile)
{
    SortedDictionary<DateTime, double> result = new SortedDictionary<DateTime, double>();
    StreamReader srTime = null;
    StreamReader srValues = null;

    try
    {
        srTime = new StreamReader(timeFile);
        srValues = new StreamReader(paramFile);

        string timeLine;
        string valueLine;

        while ((timeLine = srTime.ReadLine()) != null &&
               (valueLine = srValues.ReadLine()) != null)
        {
            // stick date on the front
            // https://learn.microsoft.com/en-us/dotnet/api/system.datetime.today
            // https://learn.microsoft.com/en-us/dotnet/api/system.datetime.parse
            string dateTimeStr = DateTime.Today.ToString("yyyy-MM-dd") + " " + timeLine.Trim();

            try
            {
                DateTime dt = DateTime.Parse(dateTimeStr);
                // https://learn.microsoft.com/en-us/dotnet/api/system.convert.todouble
                double val = Convert.ToDouble(valueLine.Trim());
                result[dt] = val;
            }
            catch (FormatException)
            {

            }
        }
    }
    catch (FileNotFoundException ex)
    {
        MessageBox.Show("File not found:\n" + ex.Message,
            "Missing File", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error reading data:\n" + ex.Message,
            "Read Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
    finally
    {
        if (srTime != null) srTime.Close();
        if (srValues != null) srValues.Close();
    }

    return result;
}
```

```
// extracts data for both stations and draws two lines on the chart
private void BtnPlot_Click(object sender, RoutedEventArgs e)
{
    if (_sherkinParamFile == "" || _rochesParamFile == "")
    {
        MessageBox.Show("Please double-click a parameter in each ListBox before plotting.",
            "No Parameter Selected", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // cant plot temperature vs rainfall on same axis
    if (TxtSherkinParam.Text.ToLower() != TxtRochesParam.Text.ToLower())
    {
        MessageBox.Show("Please select the same parameter in both listboxes.");
        return;
    }

    try
    {
        string sherkinTimeFile = Path.Combine(_sherkinFolder, "Sherkin Time.txt");
        string rochesTimeFile = Path.Combine(_rochesFolder, "Roches Time.txt");

        _sherkinData = ExtractData(sherkinTimeFile, _sherkinParamFile);
        _rochesData = ExtractData(rochesTimeFile, _rochesParamFile);

        if (_sherkinData.Count == 0 || _rochesData.Count == 0)
        {
            TxtStatus.Text = "No data to plot.";
            return;
        }

        // x-axis labels from sherkin timestamps
        PlotLabels.Clear();
        foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
        {
            PlotLabels.Add(pair.Key.ToString("HH:mm"));
        }

        List<double> sherkinValues = new List<double>();
        foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
        {
            sherkinValues.Add(pair.Value);
        }

        List<double> rochesValues = new List<double>();
        foreach (KeyValuePair<DateTime, double> pair in _rochesData)
        {
            rochesValues.Add(pair.Value);
        }

        LineSeries sherkinSeries = new LineSeries
        {
            Title = "Sherkin Island",
            Values = new ChartValues<double>(sherkinValues),
            PointGeometrySize = 6,
            StrokeThickness = 2
        };

        LineSeries rochesSeries = new LineSeries
        {
            Title = "Roches Point",
            Values = new ChartValues<double>(rochesValues),
            PointGeometrySize = 6,
            StrokeThickness = 2
        };
```

```
        ChartPlot.Series = new SeriesCollection { sherkinSeries, rochesSeries };

        if (ChartPlot.AxisX.Count > 0)
            ChartPlot.AxisX[0].Labels = PlotLabels;

        string paramName = TxtSherkinParam.Text;
        TxtPlotTitle.Text = "Parameter Chart - " + paramName;
        TxtStatus.Text = "Plotted " + _sherkinData.Count + " readings for " + paramName + ".";

        // maxelement is the smaller count
        int maxElement = _sherkinData.Count;
        if (_rochesData.Count < maxElement)
            maxElement = _rochesData.Count;

        SldCount.Minimum = 1;
        SldCount.Maximum = maxElement;
        SldCount.Value = 1;
        SldCount.IsEnabled = true;
        TxtCount.Text = "1";
        TxtCountRange.Text = "1 to " + maxElement;

        BtnRange.IsEnabled = true;
        MenuWriteDiff.IsEnabled = true;

        // write differences straight away
        WriteDifferencesFile();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error plotting data:\n" + ex.Message,
            "Plot Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

// updates the count label whenever the slider moves
private void SldCount_ValueChanged(object sender,
    RoutedPropertyChangedEventArgs<double> e)
{
    if (TxtCount == null) return;

    int count = (int)SldCount.Value;
    TxtCount.Text = count.ToString();
}

// takes the first N readings based on slider value and plots them on the second chart
private void BtnRange_Click(object sender, RoutedEventArgs e)
{
    if (_sherkinData.Count == 0 || _rochesData.Count == 0)
    {
        MessageBox.Show("No data available. Plot data first.",
            "No Data", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    try
    {
        int count = (int)SldCount.Value;

        List<DateTime> sherkinKeys = new List<DateTime>();
        List<double> sherkinValues = new List<double>();
        int n = 0;
        foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
        {
            if (n >= count) break;
            sherkinKeys.Add(pair.Key);
```

```
            sherkinValues.Add(pair.Value);
            n++;
        }

        List<double> rochesValues = new List<double>();
        n = 0;
        foreach (KeyValuePair<DateTime, double> pair in _rochesData)
        {
            if (n >= count) break;
            rochesValues.Add(pair.Value);
            n++;
        }

        RangeLabels.Clear();
        for (int i = 0; i < sherkinKeys.Count; i++)
        {
            RangeLabels.Add(sherkinKeys[i].ToString("HH:mm"));
        }

        LineSeries sherkinRange = new LineSeries
        {
            Title = "Sherkin Island",
            Values = new ChartValues<double>(sherkinValues),
            PointGeometrySize = 6,
            StrokeThickness = 2
        };

        LineSeries rochesRange = new LineSeries
        {
            Title = "Roches Point",
            Values = new ChartValues<double>(rochesValues),
            PointGeometrySize = 6,
            StrokeThickness = 2
        };

        ChartRange.Series = new SeriesCollection { sherkinRange, rochesRange };

        if (ChartRange.AxisX.Count > 0)
            ChartRange.AxisX[0].Labels = RangeLabels;

        string paramName = TxtSherkinParam.Text;
        TxtRangeTitle.Text = "Range Chart - " + paramName + " (first " + count + " readings)";
        TxtStatus.Text = "Range: showing " + count + " of " + _sherkinData.Count + " readings.";
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error displaying range:\n" + ex.Message,
            "Range Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

// finds matching timestamps between both stations and writes the differences to a txt file
private void WriteDifferencesFile()
{
    StreamWriter sw = null;

    try
    {
        List<DateTime> commonKeys = new List<DateTime>();
        foreach (KeyValuePair<DateTime, double> pair in _sherkinData)
        {
            if (_rochesData.ContainsKey(pair.Key))
            {
                commonKeys.Add(pair.Key);
            }
        }
```

```
            if (commonKeys.Count == 0)
            {
                TxtStatus.Text = "No matching timestamps to compute differences.";
                return;
            }

            string paramName = TxtSherkinParam.Text;
            string diffFile = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
                paramName + "_Differences.txt");

            sw = new StreamWriter(diffFile);
            sw.WriteLine("DateTime\tSherkin\tRoches\tDifference");

            for (int i = 0; i < commonKeys.Count; i++)
            {
                DateTime key = commonKeys[i];
                double sVal = _sherkinData[key];
                double rVal = _rochesData[key];
                double diff = sVal - rVal;

                sw.WriteLine(key.ToString("yyyy-MM-dd HH:mm") + "\t" +
                    sVal + "\t" + rVal + "\t" + diff.ToString("F2"));
            }

            TxtStatus.Text = "Differences written to " + Path.GetFileName(diffFile) + ".";
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error writing differences:\n" + ex.Message,
                "Write Error", MessageBoxButton.OK, MessageBoxImage.Error);
        }
        finally
        {
            if (sw != null) sw.Close();
        }
    }

    // menu option to write the differences file
    private void MenuWriteDiff_Click(object sender, RoutedEventArgs e)
    {
        WriteDifferencesFile();
    }

    // closes the application
    // https://learn.microsoft.com/en-us/dotnet/api/system.windows.application.shutdown
    private void MenuExit_Click(object sender, RoutedEventArgs e)
    {
        Application.Current.Shutdown();
    }
  }
}
```

4.3 App.xaml

```
<Application x:Class="WeatherStation.App"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        StartupUri="MainWindow.xaml">
  <Application.Resources>
  </Application.Resources>
</Application>
```

4.4 App.xaml.cs

```
using System.Windows;

namespace WeatherStation
{
  public partial class App : Application
  {
  }
}
```